QCon
全球软件开发大会

InfoQ 技术大会

# 从“能审”到“审出闭环”
# AI Code Review 规模化实践

李阳杰

腾讯音乐 高级测试开发工程师



# 个人介绍



## 李阳杰

腾讯音乐集团 / 研发效能中心 / 高级测试开发工程师

长期从事研发效能体系建设与AI工程化落地工作，拥有11年以上研发效能平台建设经验。现为TME/酷狗研效中心持续交付平台与AI效能专项主要负责人，主导了公司级持续交付平台及AICR代码审查平台从0到1的建设，并推动其在多条核心业务线落地。擅长将AI技术与研发流程深度融合，构建“发现-修复-验证-闭环”的质量内建体系，在保障代码质量的同时大幅提升研发效率。对AI Agent、大模型工程化、DevOps工具链有丰富的实战经验。

QCon
全球软件开发大会

InfoQ 技术大会

# 目录


01 Vibe Coding时代，Code Review的质量挑战

02 让AI从“看代码”到“懂业务”

03 多Agent与增量：放大审查效果，完成工具层闭环

04 集成交付与反思闭环：让审查结果真正落地

05 自动化节点与AICR双向联动及ACP编排

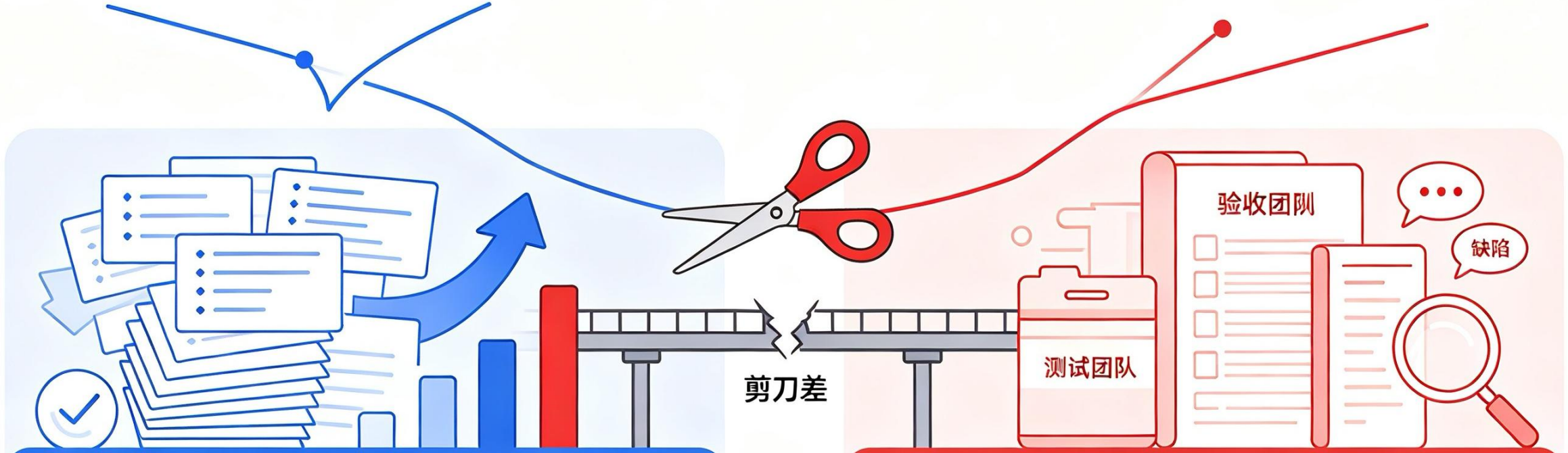
06 Coding时代质量保障体系的未来演进与规模化落地

01

# Vibe Coding时代 Code Review的质量挑战

# 代码生产力与验收能力的剪刀差

**核心矛盾：代码验收能力远远落后于代码生产力**




## 代码生产力

- AI Coding 加速产出，迭代周期缩短，交付量上升

## 验收能力

- 人工复核压力增加，测试覆盖不足，缺陷发现滞后

# 现状：三种AI审查形态的断裂

## 形态一 Prompt直调LLM

- 将Diff直接给LLM评审输出结果
- 无上下文、无结构化流程
- 结果不可复现、无度量基准
- 依赖个人经验，团队难以统一

→ 极简、廉价但粗糙

## 形态二 AI Coding内嵌

- Copilot/Cursor等IDE内嵌审查
- 实时反馈但缺乏全局视角
- 审查标准不透明、效果无统计
- 无法与团队流程规范对齐

→ 左移但不可度量

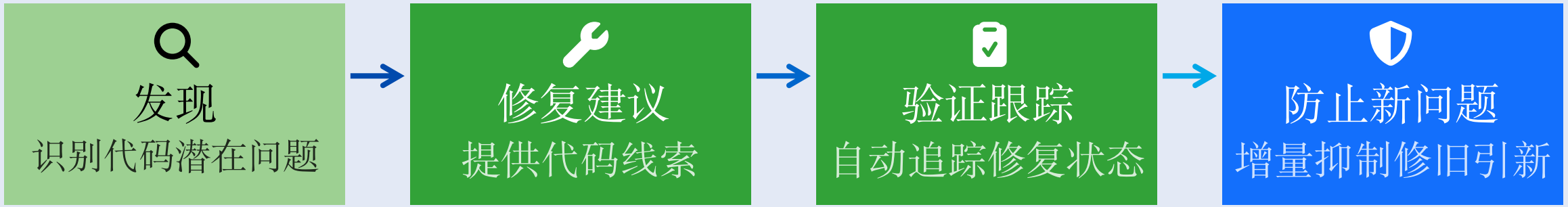
## 形态三 流水线后置扫描

- CI中集成静态分析/LLM扫描
- 生成报告后缺乏后续动作
- 问题是否修复需人工确认
- "发现即止"——报告成"死文档"

→ 有发现但无追踪

核心诉求：审查必须从"发现"走向"闭环"

# AICR以点破面：构建能闭环的质量债审查体系




## AICR 常见形态

- Prompt直调LLM—极简但极粗糙
- AI Coding内嵌—可用但不可度量
- 流水线后置—有发现但无追踪

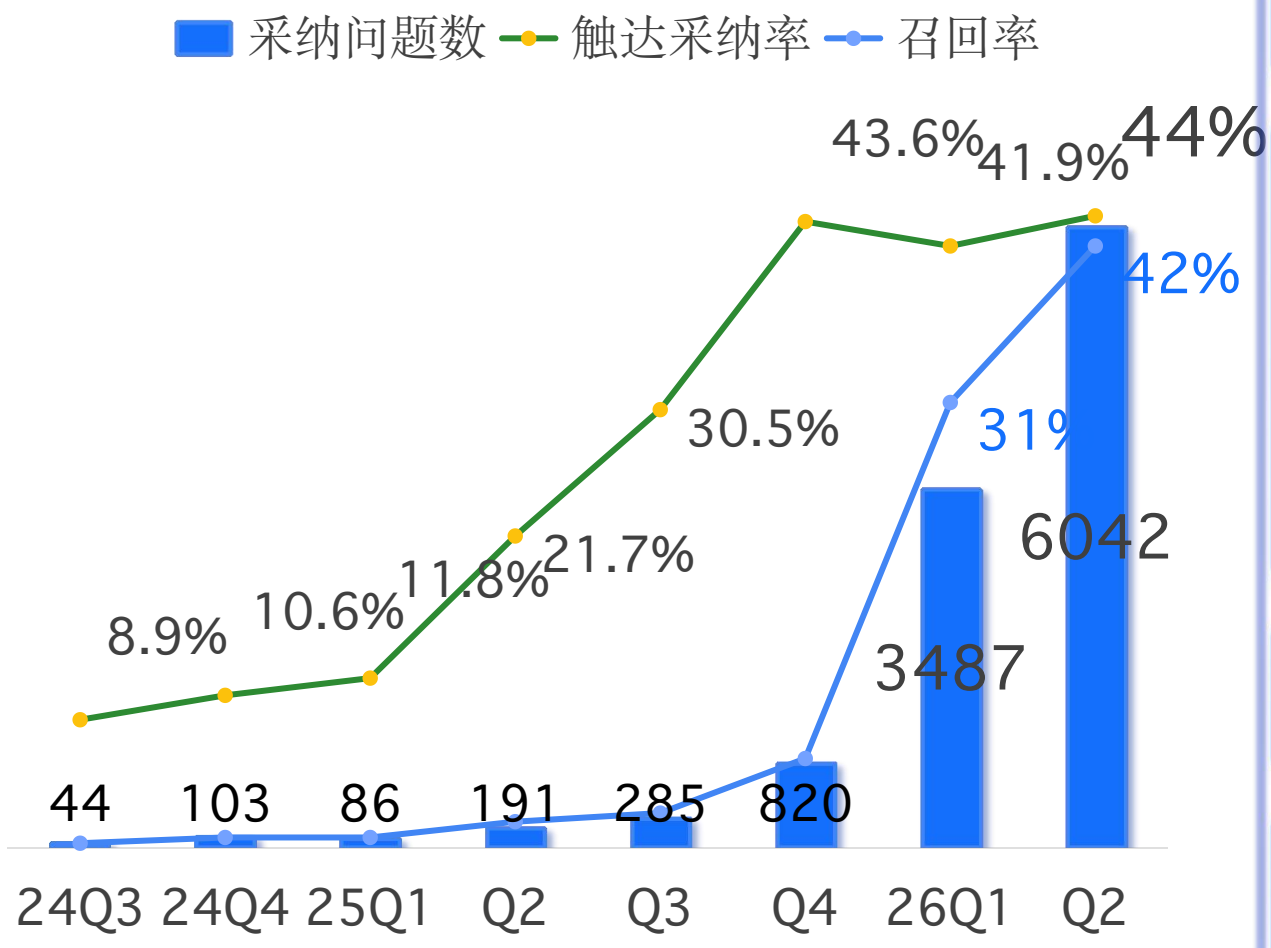
## AICR 落地为载体，体系兜底质量：

- 嵌入审查全链路
- 持续闭环的质量内建
- 根因驱动的自演进体系

从“能审”到“审准、审得起、审出闭环”

# 技术架构全景速览

## AICR效果




L1 事件入口

GitLab MR Webhook | Cnb PR 云构建 | JSON-RPC 调用 | 开放 API / 龙虾系Skill | CI 门禁 / 超时

MTP / KCD / Tapd 研效任务

L2 接入网关

事件去重与串单 | Celery 异步队列 | GitLab / PR 回调 Hook

L3 AICR 核心引擎

决策与编排

路由 / 模型选择 | Skill 管线编排 | Orchestration扇出配置

多智能体执行

主审查 Agent | 子智能体集群 | 子任务扇出调度

MCP 动态调度

工具注册与描述 | 仓库 FS / 行级读取 | 按需工具切换

数据面

事件会话记录 | 事件血缘关系 | 人设 / Skill / Fewshot 知识库

L4 多源上下文

Git 仓库 / 分支 / Diff | TAPD / MTP 需求 / 用例 | 调用图与影响面 | 监控日志 / 运行时线索

L5 交付与集成

评论与状态回写 | 指标统计亮化 | 质量门户同步 | 可视化报告页

L6 业务问题闭环

人工闭环 SKILL自动化

遗漏问题收集 → 反推分析 → 配置与模型迭代

SELF-IMPROVEMENT 规划中

读取 Skill / 提示词 / 工具描述现状 → 生成评测集（合成或真实会话沉淀） → 反思式优化器：读执行轨迹，定向改失败根因（GEPA 思路） → 候选变体 → 评测打分（轨迹回灌优化器） → 约束门：pytest 全绿、体量上限、缓存期语义不退化 → 择优合入：仅经 PR 人审，回到 L3 数据面 / Skill 库

LLM 智能决策 + MCP 动态调度 + 多智能体协同 + CI/CD 深度集成 + 业务闭环优化

02

# 让AI从“看代码”到“懂业务”

# 审查前：动态上下文拓展

```
--- a/src/main/java/com/kugou/fanxing/admin/support/txmusic/NewScreenShotManager.java
+++ b/src/main/java/com/kugou/fanxing/admin/support/txmusic/NewScreenShotManager.java
@@ -459,11 +459,11 @@ public class NewScreenShotManager {
        fxData.setType(monitoringData.getType());
        picAgentExtraData.setFxData(fxData);
        String[] labels         = label.split("@");
-       Map risk = Maps.newHashMap();
-       risk.put("Scene",monitoringData.getRiskType());
+       Map<String,String> risk = Maps.newHashMap();
+       risk.put("Scene","imageTerror");
        risk.put("Label",labels[0]);
        risk.put("SubLabel",labels.length>1?labels[1]:"");
-       risk.put("Score",monitoringData.getScore());
+       risk.put("Score",String.valueOf(monitoringData.getScore()));
        picAgentExtraData.setAlgohitrecord(Lists.newArrayList(risk));
        picAgentReq.setExtra_info(JsonUtils.toJsonString(picAgentExtraData));
        if(config.getBooleanProperty("fx.safe.ml.danger.pic.use.agent",false)){
```

拓展

```
## File: 'src/main/java/com/kugou/fanxing/admin/support/txmusic/NewScreenShotManager.java'
__new hunk__
1 +++ b/src/main/java/com/kugou/fanxing/admin/support/txmusic/NewScreenShotManager.java
__old hunk__
--- a/src/main/java/com/kugou/fanxing/admin/support/txmusic/NewScreenShotManager.java

@@ -456,15 +456,15 @@ public class NewScreenShotManager {
__new hunk__
  { # packages and imports } # 头部填充

456          fxData.setRoomId(monitoringData.getRoomId()); #上文填充
457          fxData.setSupplier(monitoringData.getSupplier());
458          fxData.setBusiness(ML_SCREEN_SHOT_DANGER);
459          fxData.setType(monitoringData.getType());
460          picAgentExtraData.setFxData(fxData);
461          String[] labels         = label.split("@");
462 +        Map risk = Maps.newHashMap();
463 +        risk.put("Scene","imageTerror");
464          risk.put("Label",labels[0]);
465          risk.put("SubLabel",labels.length>1?labels[1]:"");
466 +        risk.put("Score",String.valueOf(monitoringData.getScore()));
467          picAgentExtraData.setAlgohitrecord(Lists.newArrayList(risk));
468          picAgentReq.setExtra_info(JsonUtils.toJsonString(picAgentExtraData));
469          if(config.getBooleanProperty("fx.safe.ml.danger.pic.use.agent",false)){
470              kafkaProducer.picAgentReqTopic(picAgentReq,monitoringData); # 下文填充
__old hunk__
        fxData.setRoomId(monitoringData.getRoomId());
        fxData.setSupplier(monitoringData.getSupplier());
        fxData.setBusiness(ML_SCREEN_SHOT_DANGER);
        fxData.setType(monitoringData.getType());
        picAgentExtraData.setFxData(fxData);
        String[] labels         = label.split("@");
-       Map risk = Maps.newHashMap();
-       risk.put("Scene",monitoringData.getRiskType());
-       risk.put("Label",labels[0]);
        risk.put("SubLabel",labels.length>1?labels[1]:"");
-       risk.put("Score",monitoringData.getScore());
        picAgentExtraData.setAlgohitrecord(Lists.newArrayList(risk));
        picAgentReq.setExtra_info(JsonUtils.toJsonString(picAgentExtraData));
        if(config.getBooleanProperty("fx.safe.ml.danger.pic.use.agent",false)){
            kafkaProducer.picAgentReqTopic(picAgentReq,monitoringData);
```

## 语法树解析

切片/分组/归类

路由不同策略

## 动态扩充

Token变更规模

头/语法/前后

## Patch重组

增减分离/行号压

缩摘要/过滤

# 审查中：MCP调度动态拉取需求文档、仓库文件、运行时数据

基础建设贯穿25年周期，采纳率 11.8% > 43.6%

### 问题：仅看diff的局限

丢失需求背景与历史变更上下文，导致误判和漏判

模型无法感知业务意图，只能做纯语法层面的分析

### 解法：MCP按需调用

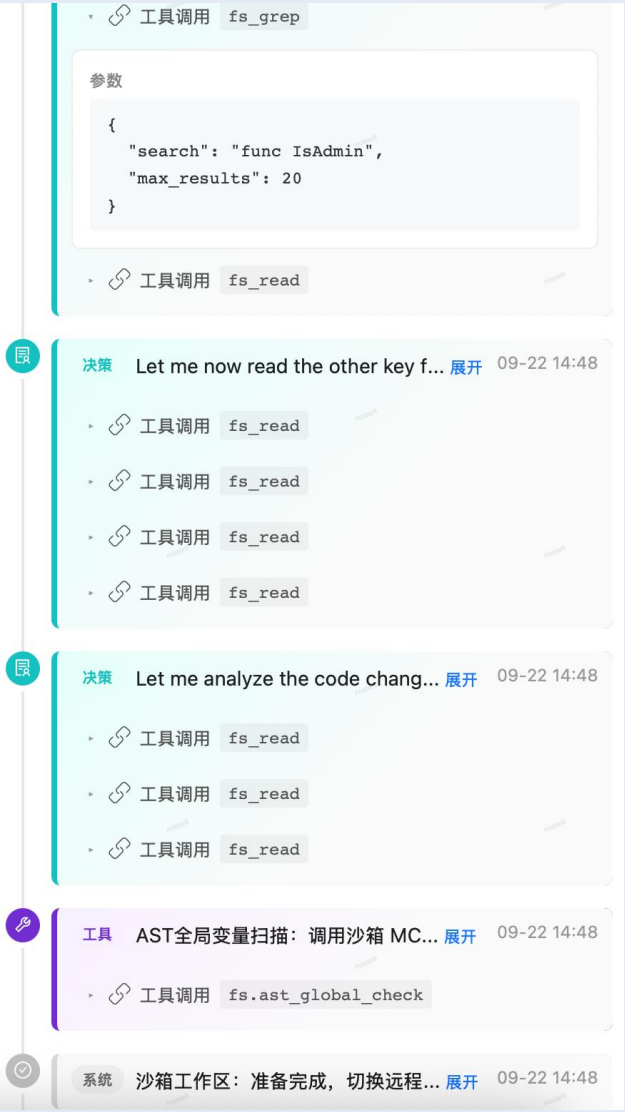
审查过程中按需通过MCP调用工具

拉取需求文档、仓库文件、运行时数据

| 能力矩阵 | 说明 | 示例 |
| --- | --- | --- |
| Git上下文 | 仓库内文件与分支视图 | POSIX沙箱标准文件读写 |
| 需求关联 | 对接需求管理系统 | TAPD / BUG等变更背景 |
| 调用与结构 | 辅助理解变更影响面 | 结合AST能力与评审策略 |
| 监控与日志 | 扩展运行时侧线索 | 接入MCP服务扩展 |


```
{
  "internal/testhelper/config.go:90": "testhelper.KugoConfigs (映射) [不安全赋值]",
  "models/sharding.go:134": "models.shardingConfigMap (映射) [不安全赋值]",
  "models/sharding.go:136": "models.shardingConfigMap (映射) [不安全赋值]",
  "service/biz/auth/rbac_service.go:31": "auth.rbacService (指针) [不安全赋值]",
  "service/biz/ticket/ticket_category_impl.go:57": "ticket.ticketCategoryMap (映射) [不安全赋值]",
  "service/biz/ticket/ticket_category_impl.go:61": "ticket.ticketCategoryMap (映射) [不安全赋值]",
  "service/biz/ticket/ticket_impl.go:1286": "ticket.showTicketMemoryCache (映射) [不安全赋值]",
  "service/biz/ticket/ticket_impl.go:469": "ticket.performanceMemoryCache (映射) [不安全赋值]",
  "service/external/piaoniu/api_limit.go:73": "piaoniu.piaoniuRateLimiter (指针) [不安全赋值]",
  "service/local/local_cache.go:102": "local.cacheInstance (映射) [不安全赋值]",
  "service/local/local_cache.go:118": "local.cacheInstance (映射) [不安全赋值]",
  "service/monitor/monitor.go:77": "monitor.KgmSrvInst (指针) [不安全赋值]",
  "service/monitor/monitor.go:90": "monitor.KgmSrvInst (指针) [引用全局指针]",
  "service/rpc/circuit_breaker.go:79": "rpc.tmeCircuitBreaker (指针) [不安全赋值]",
  "service/rpc/goods_media_store.go:52": "rpc.goodsMediaStoreClient (指针) [不安全赋值]",
  "service/rpc/kugou_pid.go:68": "rpc.kuGouClientCodePidMap (映射) [不安全赋值]",
  "service/rpc/kugou_pid.go:71": "rpc.kuGouClientCodePidMap (映射) [不安全赋值]",
  "service/rpc/private_msg_limit.go:38": "rpc.privateMsgSendRateLimiter (指针) [不安全赋值]",
  "service/rpc/ticket_activity_test.go:337": "rpc.lotteryActivityFetchGroup (指针) [不安全赋值]",
  "service/rpc/ticket_activity_test.go:339": "rpc.lotteryActivityFetchGroup (指针) [不安全赋值]",
  "service/rpc/tme_safety.go:214": "rpc.whiteCatBreakerInstance (指针) [不安全赋值]",
  "service/rpc/tme_singer.go:77": "rpc.tmeSingerMappingServiceInstance (指针) [不安全赋值]",
  "service/rpc/white_cat_degrade_test.go:24": "rpc.whiteCatBreakerInstance (指针) [不安全赋值]",
  "service/rtconf/gray.go:76": "rtconf.grayToolsMap (映射) [不安全赋值]",
  "utils/priority_submit_verify.go:54": "utils.rpcHelpers (映射) [不安全赋值]",
  "utils/priority_submit_verify.go:56": "utils.rpcHelpers (映射) [不安全赋值]",
  "utils/priority_submit_verify.go:60": "utils.rpcHelpers (映射) [不安全赋值]"
}
```


全局变量AST



# 多维配置驱动注入




## 无上下文

看到了一个正确的页面跳转

→ AI判定：**没问题！** 逻辑没语法缺陷

```
95  +       if (info?.roomId) {
96  +         window.location.href = `kuqunapp://start.app?type=20&roomId=${info.roomId}`;
97  +       }
```

## 有上下文（需求+仓库上下文+动态读）

歌房跳转使用硬编码 URL Scheme而非设计要求的 openRoom(roomId) 桥接方法，无法保障多端的正确跳转

→ AI判定：**高危潜在问题 高置信度**

设计文档明确要求/仓库为跨端应用/Grep证实/历史含同类Bug

03

# 多Agent与增量

放大审查效果，完成工具层闭环

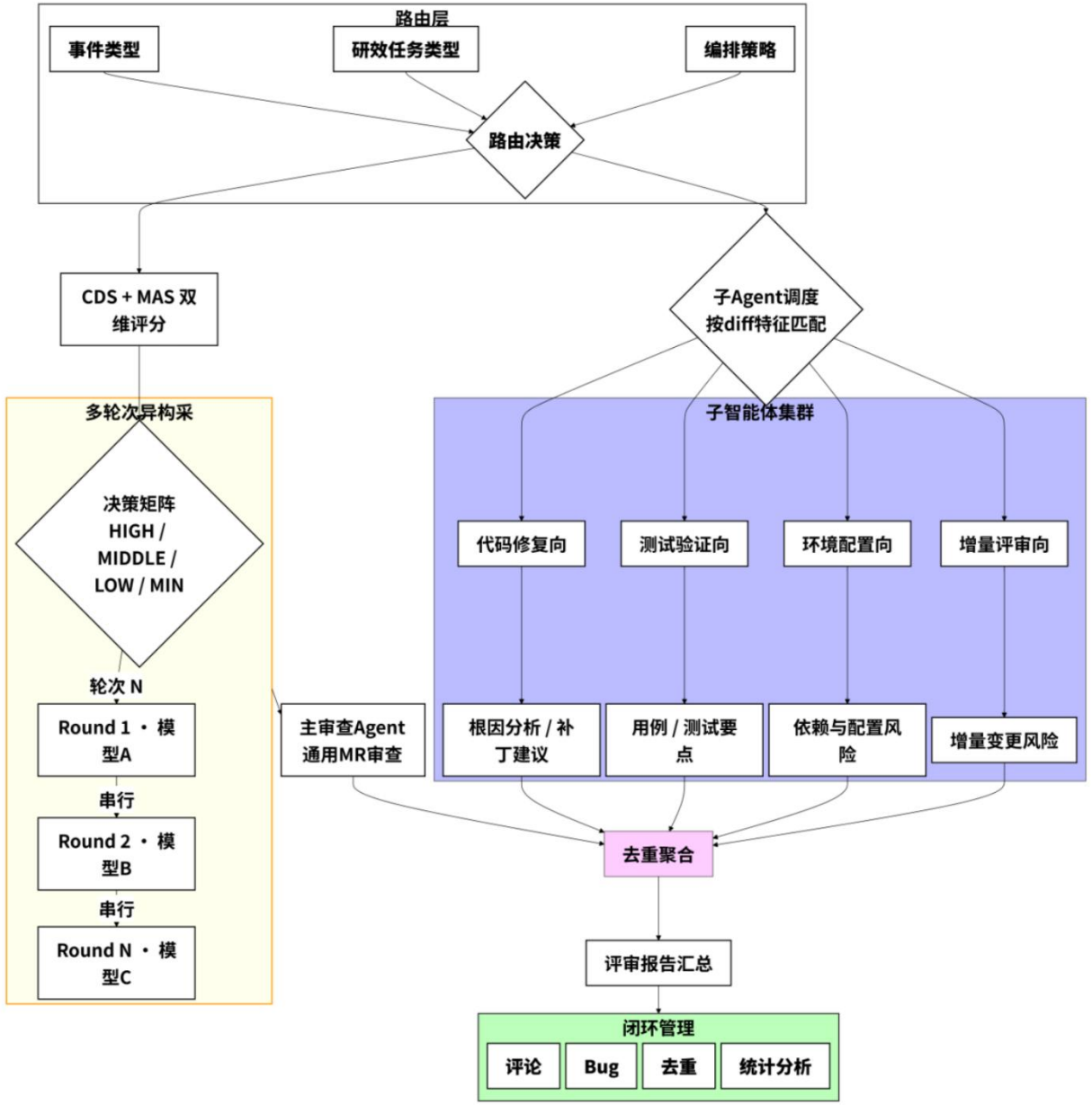
# 异构多轮次采样

## 多轮次异构采样

- 不同模型（DeepSeek → Claude → Gemini），视角互补
- 采样数由 CDS（开发者贡献分）+ MAS（模块关注分）决策
- 高价值MR深度多轮审查，低风险MR单轮快速通过
- 各轮独立产出 → 去重聚合，信噪比显著提升
- Token消耗与审查价值正相关，按需匹配成本

## 多智能体异质分工

- 每个Agent聚焦特定领域，不做重复劳动
- 专项Agent拥有领域知识和专用Prompt
- 输出独立可统计，效果可度量
- 按需调用，非必要不激活
- 减少同质化重复Token，信噪比大幅提升



# 异质垂类分工

## 通用审查专家 * 多轮采样 * 子专家矩阵

通用MR代码审查，覆盖全量变更分析

### 垂类审查专家

#### 数字类专项Agent

64位ID、计算、精度、溢出等数字相关

**垂类高优占比：35%**

#### Sonar智能审查

传统Sonar审查与AI双向联动强化审查

**采纳率：87%**

#### 部署归因Agent

明确部署代码侧归因快速推动ops恢复

**部署修复：1min**

### 辅助Agent

#### 闭环追踪Agent

自动追踪历史评审问题的修复状态，与GitLab/TAPD联动

**修复跟踪率：93.1%**

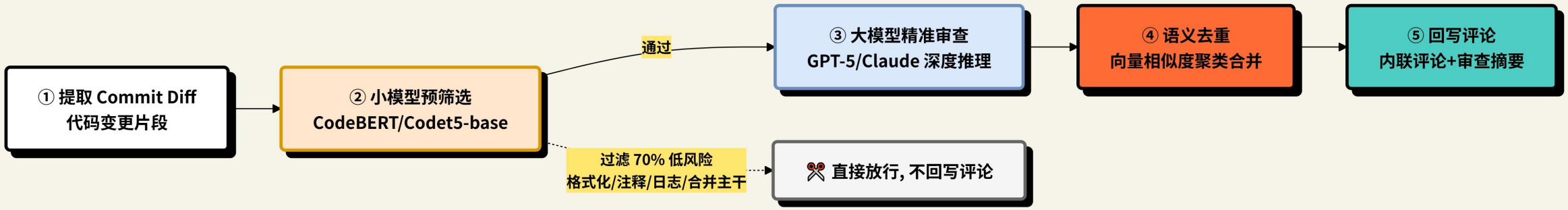
#### 增量评审Agent

修复提交后立即扫描，验证是否引入新风险

**新问题发现率：33%**

减少同质化重复token，专家效果输出可独立统计与运营

# 增量评审




## 01

小模型预筛选 —“降本”

- 输入：Diff + 文件元信息
- 任务：缺陷引入概率评分
- 辅助：过滤从主干合入/注释等
- 效果：大模型调用量成本骤降

## 02

大模型精准审查 —“准确”

- MCP决策深度审查
- 动态上下文增强
- 审查维度：逻辑错误、边界条件、安全隐患、性能热点

## 03

语义去重 —“降噪”

- 问题：多处修改触发重复问题
- 方法：向量化、LLM聚类召回
- 合并同类问题，择优合并输出
- 结果：过滤40%额外噪音

## 核心收益

| | |
|---|---|
| 大模型调用量 | ↓ 70% |
| 问题噪音几率 | ↓ 40% |
| 高危问题召回 | ↑ 21% |

# 路由与编排

多轮次采样路由

## CDS + MAS 动态决策轮次

CDS开发者贡献分+ MAS模块关注分 双维评分

HIGH / MIDDLE / LOW / MIN 四档动态轮次

每轮固定不同模型，串行异构互补

子Agent按需调度

## Diff 特征匹配专项 Agent

按变更匹配专项 Agent，非必要不激活

抑制"所有 Agent 全开"造成的评审风暴

输出独立可统计，效果可度量

灰度控制与回滚

## 四阶段渐进 + 秒级回滚

全局开关，≤1 即关闭

仓库级 orchestration.yaml 独立配置

从 1 轮逐步放开至 5 轮，数据驱动决策

失败容错

## 多供应商 + 链式重试

DeepSeek / Kimi / GLM 多供应商负载分散

单供应商故障时自动链式重试

99.9% 系统可用性

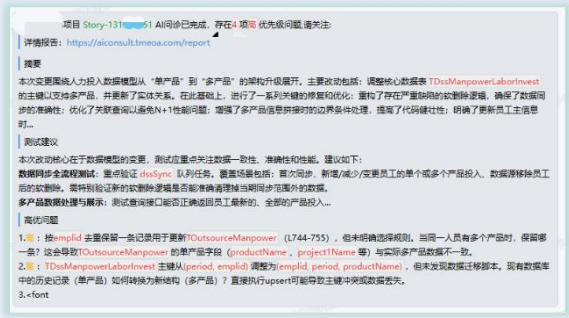
04

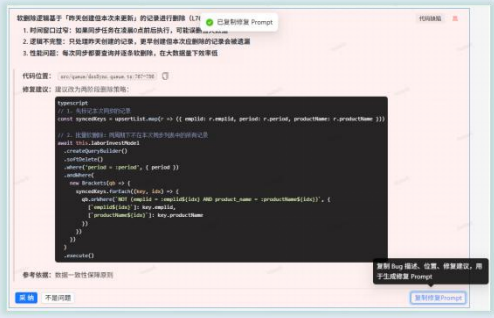
# 集成交付与反思闭环

让审查结果真正落地

# 双Agent联动闭环流程



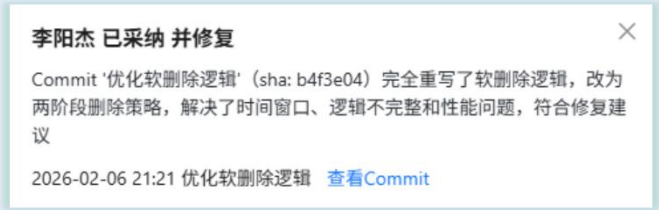


















修复召回率 71%

追踪发现问题 33%

# 数据驱动改进

## 数据收集

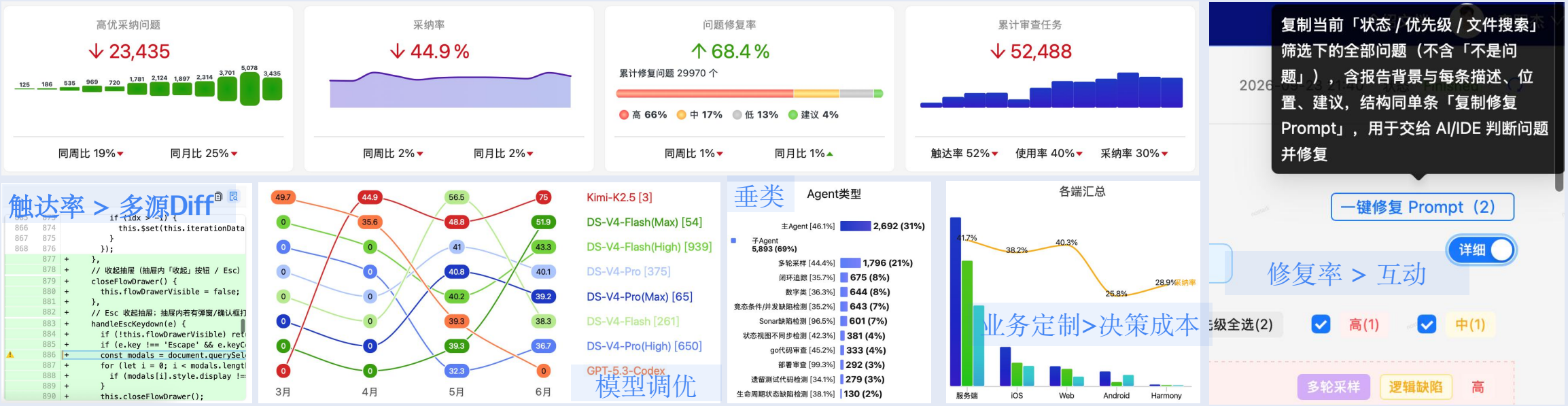
- 评论中提供「一键采纳」修复建议
- 「一键忽略误报」按钮
- 反馈数据进入自学习系统
- 误报标记持续优化模型输出

## 数据闭环

- 采纳行为 → 正向反馈
- 拒绝行为 → 负向反馈
- 忽略行为 → 中性反馈
- 全部回流至效果大盘

## 效果大盘

- 触达采纳率实时追踪
- 修复跟踪率趋势分析
- 模型效果对比看板
- Agent维度下钻分析



# Good Case：典型高价值案例

## 性能与稳定性

高危

- goroutine泄漏 — 并发资源未释放，导致服务OOM
- 热路径性能退化 — 高频率调用链路上的低效实现
- 这些问题在集成测试中难以暴露，但diff审查可前置发现

## 安全/越权/数据隔离

高危

- 权限检查绕过 — 关键校验被注释或条件覆盖
- 数据隔离破坏 — 跨租户数据访问边界缺失
- 安全类问题一旦上线，影响面极大

## 边界与配置

中高危

- 远程开关夹紧 — 配置变更未做边界校验
- 数值溢出 — 整数运算未考虑最大值场景
- 异常配置组合在Mock环境下难复现

## 并发/Panic风险

高危

- 类型断言Panic — 接口转换未做ok检查
- 竞态条件 — 共享状态未加锁保护
- 并发问题在单测中极难稳定复现




沉淀优秀案例，记忆/权重提高缺陷发现概率

# Bad Case：Bug 根因分析与 Agent 自沉淀

## 自演进飞轮

审查发现问题 → 遗漏到后环节 → 根因分析沉淀 → 增强审查能力

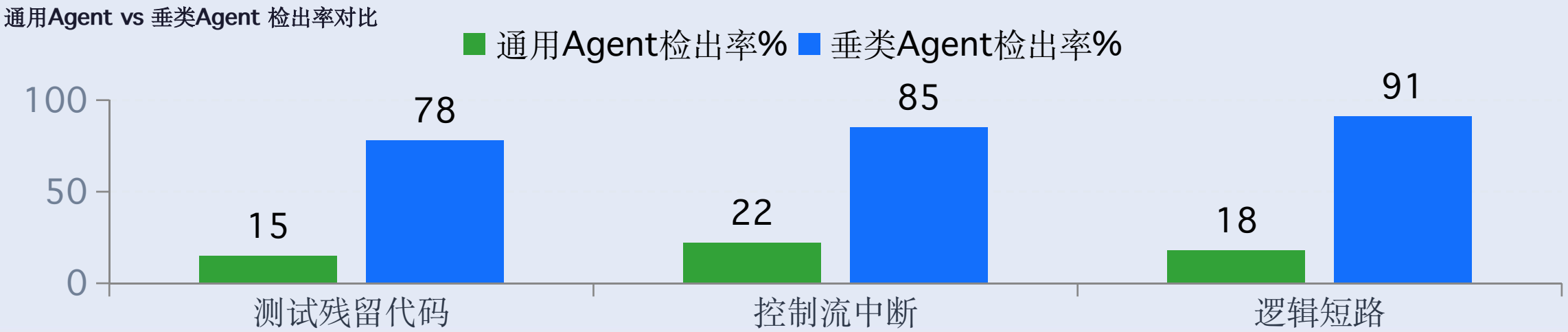
每一次遗漏都成为体系进化的养分，审查能力随时间持续增强，而非静态不变

正向反馈闭环










每一次遗漏，都是垂类专家进化的养分

05

# 自动化节点与AICR双向联动及ACP编排

# AICR与测试经典金字塔

**AICR赋传统自动化智能，经典测试体系拓展审查边界**



# 静态扫描智能评审

## 双向印证代码质量

### CICD触发「代码检测」节点

按仓库配置调起 SonarQube 扫描

### 自动化扫描结果

走查健康度：Blocker / Critical / Major 与增量 diff 文件健康分

### 触发AICR回调

异步回调 AICR 的审查接口，异常只记日志

### AICR固化落库清洗去重

增量问题落库 ScanIssue，带分支与行级 SCM 血缘归属

### Sonar 子 Agent 逐条评审

误报判定 · 严重性分级 · 中文描述与修复建议

### 生成标准 AICR 问题

并入现有报告体系，支持状态标记与效果追踪

pipeline/trigger_review.py · KCD

```python
def trigger_review(ctx, stage, force=False):
    # 未明确取消勾选，就自动触发 AI 审查
    if force or (ctx.team.auto_review
                 and not has_history(ctx)):
        # 审查依赖静态扫描数据：本次流水线没有扫描节点就静默补跑
        if not ctx.repo.has_scan_node():
            try:
                run_scan(ctx)
            except Exception:
                logger.exception('run_scan failed')    # 不阻塞流水线
        dispatch(task_review, ctx.id, stage)
```

严重性映射SonarQube → AICR

| | | |
|---|---|---|
| BLOCKER → 高 | CRITICAL BUG → 高 | CRITICAL SMELL → 中 |
| MAJOR → 中 | MINOR / INFO → 低 | 测试文件降级 |

CICD 视角 · 确定性口径

AICR 视角 · 问题改得动

# 静态扫描智能评审技巧

## 保留真正关注的问题

false_positive 标记 + 判定理由 + 规则依据，逐条过筛后再进报告

降噪

## 明确规则AICR双向验证

认知复杂度 16 / 15、重复字面量规则自然语义结合场景判断

召回

## 可读、可执行建议

描述、建议、参考依据一次给全，复用修复建议精确提效

提效

## 仓库/分支/时间趋势

UNIQUE · 分支 · 已修复标记，能按分支与时间翻回

可追溯

### SonarQube 规则命中

规则边界清晰的缺陷（复杂度、重复字面量、明确空指针）判定干脆，直接采信并归档

### AICR 语义补位

规则说不清的语义类缺陷，交回主审查 Agent，用 diff + 仓库文件 + MCP 上下文补位

## 口径 · 唯一约束

UNIQ(repo,path,rule,hash)

## 产物 · 归档

同一份报告，同一套状态

## 闭环 · 追踪

再次走查即可判定 已修复

## 降噪实绩 · 高等级

607 条收敛 8类误报模式

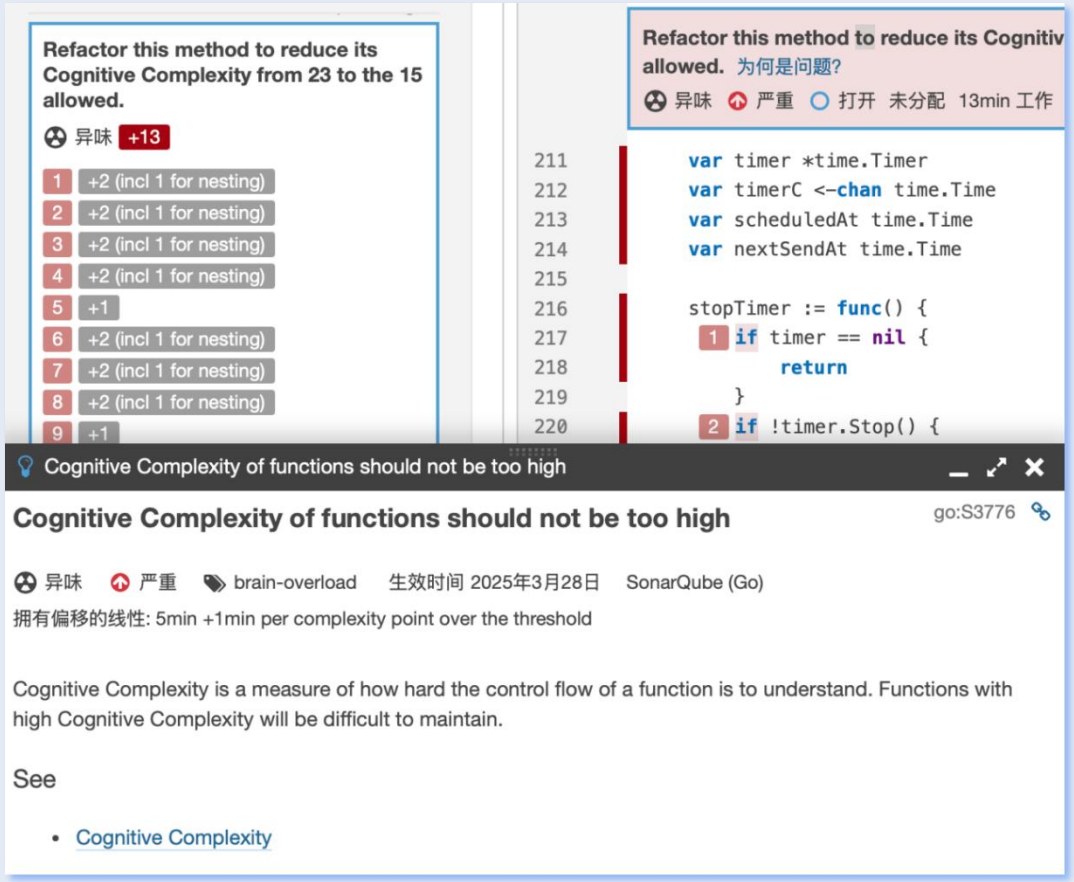
### 过去 · 裸用 SonarQube

英文规则名 + 行号，问题一屏刷不完，没人排优先级；扫描报告与审查结果两张皮

### 现在 · 智能评审

误报先筛、严重性分级、中文描述与修复建议，并入同一份报告与同一套状态

# 静态扫描智能评审案例




解释性 · 确定性

智能化 · 96.5%采纳

函数 `probeURL` 的认知复杂度为 33，远超允许的 15，内部包含无限循环、限速等待、HTTP 请求构建、重试策略和多个提前返回路径，理解难度大

代码规范 高

```
92 92  // probeURL 执行一个 URL 的完整尝试循环。每次物理请求前都消耗限速令牌，网络
93 93  // 错误和可重试响应按 retry.Policy 退避，context 取消则把任务标记为未完成。
94 94  func (c *FileClient) probeURL(ctx context.Context, rawURL string) (result fileProbe
95 95          for attempt := 0; ; attempt++ {
96 96                  if err := c.limiter.Wait(ctx); err != nil {
97 97                          result.FailureReason = fileFailureContextCanceled
98    -                     result.FailureDetail = probeErrorDetail(err)
99 98                          return
100 99                 }
101 100                request, err := http.NewRequestWithContext(ctx, http.MethodGet, raw
102 101                if err != nil {
103 102                        result.Completed = true
104 103                        result.FailureReason = fileFailureInvalidRequest
105 104                        return
106 105                }
107 106                request.Header.Set("Range", fmt.Sprintf("bytes=%d-%d", c.rangeStart
108 107                // 每次循环都重新构造带 context 的请求，保证重试不会复用已消费的请求体或已取消
109 108                attemptResult, retryable := c.probeRequest(request)
110 109                attemptResult.Attempts = result.Attempts + 1
111 110                result = attemptResult
112 111                if result.Valid {
113 112                        result.Completed = true
114 113                        return
115 114                }
116 115                if ctx.Err() != nil {
117 116                        result.FailureReason = fileFailureContextCanceled
118   -                     result.FailureDetail = probeErrorDetail(ctx.Err())
119 117                          return
120 118                 }
121 119                 if !retryable || attempt >= c.retry.MaxRetries {
122 120                         result.Completed = true
123 121                         return
124 122                 }
```

# 借 AI 智能，还确定性验证

## 单元测试 ✖ AICR

**自动化侧** 单测 + **增量覆盖率（diff-cover）** 守住「改完有没有崩」：后端用例 **24 → 385**，增量覆盖 **44% → 98%**

**AICR 侧** 反审AI coding产物，扫出 **4 个真问题**：回调异常被吞、恒返回成功 · 失败构建仍读残留覆盖率 · 差异解析未校验结构 · 覆盖率分子分母类型漂移

**双向互补** 合力发现“失败构建被当成成功”，逐条修复后**全量单测 283 条通过**验证无回归，且每条都AI coding补全回归用例

## 接口自动化 ✖ AICR

**自动化侧** CI 脚本在**部署完成**后**测试真实目标**，证明「这条链路真的通」——这是接口层唯一不可替代的那一步

**AICR 侧** 看出**跨端契约不一致**：后端把入参声明成**整型数组**，调用方传的却是**逗号分隔字符串**，参数绑定失败、接口直接报错

**双向互补** 接口测试管**运行期通不通**，AICR 管**契约与边界对不对**。列表页每次进入无条件发请求、没按日期缓存这类「能跑通但不对」，接口测试无感，AICR 一眼看出

| 后端用例 | 增量覆盖率 | AICR 自审一轮 | 遗漏聚类 · 接口类 |
|---|---|---|---|
| 24 → 385 | 44% → 98% | 9 条 全采纳 · 0 误报 | 契约 9 条 · 重复请求 5 条 |

## 部署失败归因

部署一失败，CICD 结算即触发独立归因子 Agent：变更文件清单 + 失败日志尾部，产物复用报告页，追踪快速修复

## 自动化给的证据 · 跑出来的

部署后验证真实目标、单测断言覆盖改动行——证明这次改动没破坏原功能，完善的回归测试支撑频繁修改

## AICR 给的证据 · 读出来的

合流前读全量 diff 与上下文——证明这次改动“本来就对”，印证动态测试结果，代码层级明确功能完善

# ACP 编排 —— 把自动化节点串成一条闭链

### 走查结果 → 智能报告

增量问题直接进 AICR 队列，误报判定与严重性用同一套口径

代码扫描

### 审查结论 → 用例补齐

单测由 AI 生成与补全；审查结论反过来指导该补哪些用例

AI单元

### 探索 → 固化 → 反哺

智测探索缺陷，固化回归用例；AICR评估覆盖，免测提效降本

AI智测

### 归因 → 追踪

部署与运行时异常关联到代码变更，归因后回到 AICR 追踪闭环

AIops

## ACP 编排层 · 统一接入与出口

业务与平台自定义的审查相关 Agent 按统一协议接入；编排顺序、上下文注入、结果归一都在 AICR 侧完成

业务 / 平台 Agent ↔ ACP 编排层 ↔ 报告 · 追踪

开放 API · API Token

JSON-RPC · review.start

仓库编排配置 · orch.yaml

配置中心 · 专家 · 产品等

## 接入即复用 · 三件套

**报告体系** — 统一问题模型与报告页，接入方的产出自带展示与流转

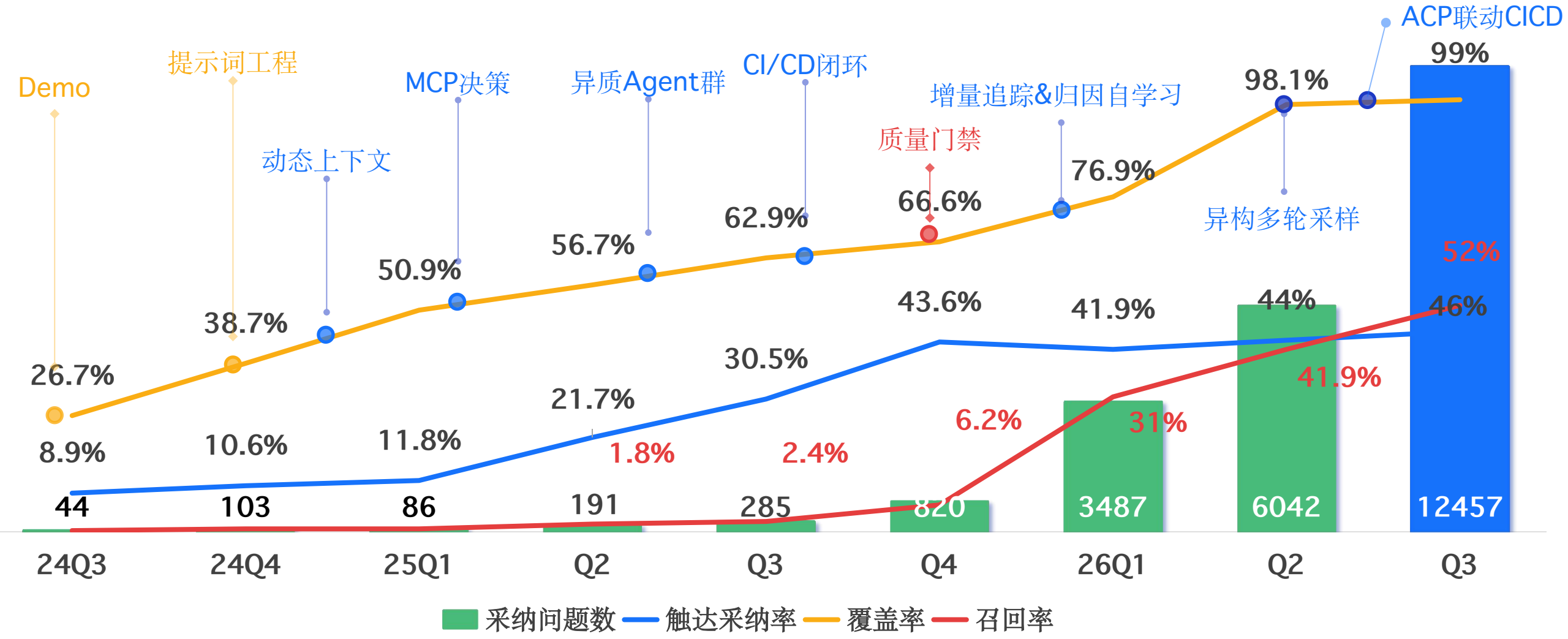
**智能序列** — 产物结构化落到同一套字段，跨 Agent 可比、可统计

**追踪闭环** — 父子事件与修复追踪复用，问题能一路跟到修复验证。跨工具、跨节点：CI 节点、IDE、业务平台、外部 Agent 汇到同一条闭环，审查不再局限于一次 MR dif

06

# Coding时代质量保障体系的未来演进与规模化落地

# 落地效果：20000+高价值问题发现，全业务线接入



# 动静结合展望

## AICR（静态视角）

- 边界条件与契约缺陷
- 配置安全与数据隔离
- 逻辑短路与控制流异常
- 代码规范与最佳实践
- 并发风险与类型断言
- 前后端通用契约

+

## 智测（动态视角）

- ◇运行时并发与竞态条件
- ◇动态配置组合验证
- ◇集成场景与端到端流程
- ◇用户体验与UI回归
- ◇性能基准与负载时表现
- ◇非预期探索性缺陷

「静+动」共享上下文的完整质量网 VS 未来趋势

研发生命周期多个Agent运行：AICR、静态扫描、自动部署、单元、接口、UAT、性能、安全、Ops、巡检、反馈.....

## 双向补齐

- 静 AICR基于代码变更范围、逻辑分支、异常路径，自动推荐应测点，注入到单元/接口/集成测试序与依赖关系
- 动 反馈真实业务操作路径、契约边界，反哺审查规则、关注点

## 相互校准

- 静 白盒验证基于通过分析改动摘要与代码Trace，判断测试点已被审查触达，辅助智测反向归因
- 动 验证AICR准确性，校准“动态”可能性问题，人工+自动化闭环

# 技术回顾

01 上下文

从单源静态文本到多源动态调度 —— MCP协议打通需求、仓库、监控等多维信息

02 形态

从同质多模型到异质多智能体分工协同 —— 各Agent专精不同领域，减少重复与冲突

03 闭环

从只报不修到MR/流水线/任务系统联动的修复闭环 —— 追踪+增量双Agent保障

04 集成

从独立外挂到CI/CD与研效平台深度集成

05 演进

从静态规则库到根因驱动的模式识别与Agent自沉淀

# AiCon 北京站

全球人工智能开发与应用大会

## 探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

### 专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

### 专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

### 专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

### 专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

### 专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

### 专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

# THANKS

李阳杰

TME 腾讯音乐集团

